САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет прикладной математики — процессов управления

ОТЧЁТ ПО НИР

Этап 2. Постановка задачи, анализ существующих решений,

выявление узких мест и план реализации

Тема: Надёжность и управление размещением в федерации Kubernetes-кластеров

на основе SRE-метрик, DR-процедур, политик, каталога и meta-scheduler'a

Студент: Трофимов И.А.

Руководитель: Корхов В.В.

Санкт-Петербург, 2025

1. Введение

Современные облачные инфраструктуры всё чаще строятся на основе федераций Kubernetes-кластеров — совокупностей независимых кластеров, объединённых единой плоскостью управления. Такой подход позволяет распределить нагрузку между несколькими датацентрами или облачными провайдерами, обеспечить географическую близость к пользователям и выполнить требования регуляторов по хранению и обработке данных в пределах определённых юрисдикций.

Вместе с тем федеративная архитектура существенно усложняет управление надёжностью: сбои в одном кластере могут затронуть другие, а ручные процедуры восстановления становятся узким местом. Отсутствие единого, формализованного механизма размещения рабочих нагрузок (workloads) с учётом актуальных SRE-метрик и политик безопасности/регулирования неизбежно приводит к деградации SLO и увеличению времени восстановления после аварий.

Данная работа направлена на разработку сквозного контура управления надёжностью и размещением в федерации Kubernetes-кластеров. Контур включает: наблюдаемость на основе SLI/SLO, автоматизированные DR-процедуры, исполняемые политики размещения и интеллектуальный meta-scheduler, выбирающий оптимальный кластер по совокупности метрик.

Настоящий отчёт соответствует второму этапу НИР: формулируется развёрнутая постановка задачи, проводится анализ существующих инструментов и подходов, выявляются их ограничения применительно к поставленной задаче, предлагаются авторские методы решения и детализируется план реализации на следующий семестр.

2. Постановка задачи

2.1. Контекст и область исследования

Федерация Kubernetes-кластеров — это набор из двух и более кластеров k8s, управляемых совместно через общую control plane (например, KubeFed v2 / Admiralty / Liqo). Каждый кластер обладает собственными ресурсами (узлы, сеть, хранилище), может располагаться в разных регионах или у разных провайдеров, и работает автономно при недоступности управляющей плоскости.

В рамках данной работы рассматривается стенд из 2–3 кластеров с единой управляющей плоскостью. Основной объект исследования — механизмы, обеспечивающие: (а) измеримость надёжности через SLI/SLO; (б) устойчивость к типовым отказам; (в) автоматическое восстановление по DR-процедурам; (г) соблюдение политик размещения; (д) осмысленный выбор кластера планировщиком.

2.2. Формулировка исследовательских вопросов

В ходе работы предстоит ответить на следующие вопросы:

1. Как корректно определить и измерить SLI/SLO для мультикластерного сервиса, агрегируя метрики из нескольких кластеров?
2. Какие конкретные механизмы (Horizontal Pod Autoscaler, трафик-шейпинг, failover) в наибольшей мере улучшают SLO при типовых сбоях (узел недоступен, сеть деградирует, кластер частично отказывает)?
3. Как обеспечить воспроизводимый DR-цикл: обнаружение сбоя → автоматический failover → управляемый подъём кластера, с фиксируемыми значениями RTO и RPO?
4. Как формализовать политики размещения workloads (ограничения по регионам, меткам безопасности, доступности ресурсов) и обеспечить их machine-readable исполнение без ручного вмешательства?
5. Как построить meta-scheduler, агрегирующий метрики качества и политики из каталога, и сравнить его эффективность с наивным (round-robin или random) выбором кластера?

2.3. Целевые показатели и критерии успеха

Результаты работы считаются достигнутыми при выполнении следующих условий:

Область	Метрика	Целевое значение

SLO / Observability	Доля успешных запросов (availability)	>= 99.0% в штатном режиме
SLO / Observability	p95 latency	< 200 мс при нормальной нагрузке
SLO / Observability	Error budget burn rate	Отслеживается, алерт при превышении
DR	Время обнаружения сбоя (TTD)	< 60 секунд
DR	Время восстановления (RTO)	< 5 минут для failover сценария
DR	Потенциальная потеря данных (RPO)	< 30 секунд при stateless нагрузке
Политики	Число нарушений политик размещения	0 после внедрения enforcement
Meta-scheduler	Снижение error rate vs наивный выбор	>= 20% в сценарии деградации

3. Анализ существующих подходов и решений

3.1. Управление федерацией Kubernetes

На сегодняшний день существует несколько подходов к организации федерации кластеров Kubernetes:

KubeFed v2 (Kubernetes Federation)

Официальный проект CNCF для федерации ресурсов. Поддерживает распространение (propagation) ресурсов (Deployments, Services, ConfigMaps) между кластерами через FederatedResource CRD. Из ограничений: сложность эксплуатации, отсутствие встроенного механизма принятия решений на основе метрик, фактически заморожен в части активной разработки с 2022 года.

Admiralty (formerly multicluster-scheduler)

Реализует мультикластерное планирование через виртуальные узлы (virtual-kubelet). Позволяет прозрачно планировать поды в удалённых кластерах. Не решает задачу policy enforcement и не имеет встроенной интеграции с SRE-метриками для принятия решений.

Liqo

Позволяет «расширять» кластер за счёт ресурсов другого кластера через peering. Хорошо подходит для burst-сценариев, но не предоставляет механизма детального контроля политик размещения и DR-автоматизации.

OCM (Open Cluster Management)

Платформа Red Hat для управления несколькими кластерами. Включает Placement API для распределения workloads по кластерам с учётом меток и ограничений. Наиболее функциональное из существующих решений, однако не предоставляет встроенной интеграции SLO-метрик в алгоритм выбора кластера и требует значительного операционного overhead.

3.2. SRE-метрики и SLO в Kubernetes

Для измерения надёжности в Kubernetes-экосистеме применяются следующие подходы:

Prometheus + Grafana

Де-факто стандарт для сбора и визуализации метрик. Prometheus реализует pull-модель сбора с поддержкой service discovery в k8s. Grafana предоставляет

дашборды. Для SLO применяются библиотеки prometheus-slo-library (Pyrra, Sloth) — они автоматически генерируют recording rules и alert rules из декларативных SLO-определений. Ограничение в контексте федерации: каждый кластер имеет свой Prometheus; агрегация требует либо Prometheus Federation, либо Thanos/Cortex/Mimir.

OpenTelemetry

CNCF-стандарт для распределённой трассировки, метрик и логов. Позволяет унифицировать инструментирование сервисов вне зависимости от бэкенда. Важен для получения end-to-end латентности запросов, проходящих через несколько кластеров.

Thanos / Cortex / Mimir

Решения для долгосрочного хранения и глобального запроса к метрикам из нескольких Prometheus-инстансов. Thanos использует механизм sidecar-компонентов и object storage. Это позволяет строить глобальный SLO-дашборд поверх данных всех кластеров федерации.

3.3. Disaster Recovery в Kubernetes

Задача DR в контексте k8s включает несколько аспектов:

Velero

Наиболее распространённый инструмент для резервного копирования и восстановления состояния Kubernetes-кластера (ресурсы etcd + Persistent Volumes через CSI). Поддерживает расписание резервного копирования и восстановление в другой кластер. Ограничение: Velero не автоматизирует failover трафика и не принимает решения на основе метрик.

Chaos Engineering (Chaos Mesh / LitmusChaos)

Инструменты для контролируемого внесения сбоев: завершение подов, сетевые задержки, отказы узлов. Необходимы для воспроизводимого тестирования DR-процедур. Обеспечивают экспериментальную базу для сравнения «до/после» внедрения механизмов надёжности.

Argo Rollouts / Flagger

Реализуют прогрессивную доставку (canary, blue-green) и автоматический rollback при деградации метрик. Частично решают задачу failover на уровне отдельного кластера, но не оперируют концепцией мультикластерного переключения трафика.

3.4. Policy Enforcement

Kyverno

Kubernetes-native движок политик, работающий как admission webhook. Политики описываются в YAML и могут запрещать создание ресурсов, мутировать их или генерировать связанные ресурсы. Хорошо интегрируется с GitOps-процессами. В контексте данной работы используется для enforcement политик размещения на уровне отдельного кластера.

OPA Gatekeeper

Также реализует admission webhook на основе OPA (Open Policy Agent) с языком политик Rego. Более гибкий и выразительный, чем Kyverno, но требует знания Rego. Подходит для сложных политик с условиями, зависящими от внешних данных.

3.5. Планирование нагрузки (Scheduling)

Стандартный kube-scheduler принимает решения только в рамках одного кластера. Для мультикластерного планирования существуют:

- OCM Placement API — декларативный выбор кластеров по меткам и ограничениям ресурсов;
- Scheduled Scaler / KEDA — масштабирование в зависимости от внешних метрик;
- Custom controllers (Operator pattern) — написание собственного контроллера размещения на базе controller-runtime.

Общий недостаток всех существующих решений: ни одно из них не объединяет в едином контуре политики размещения, актуальные SLO-метрики всех кластеров и DR-состояние кластера для принятия комплексного решения о размещении рабочей нагрузки.

4. Выявление узких мест существующих подходов

4.1. Разрозненность компонентов

Все описанные инструменты решают отдельные подзадачи: Prometheus — метрики, Velero — backup, Kyverno — политики, Admiralty — мультикластерное планирование. Отсутствует единый контур управления, в котором решение о размещении workload принималось бы с учётом всей совокупности данных: текущих SLO-метрик, политик, DR-состояния кластера.

Следствие: оператор вынужден вручную координировать несколько систем, что является источником ошибок и задержек при реагировании на инциденты.

4.2. Отсутствие метрико-ориентированного выбора кластера

Существующие federation schedulers (OCM Placement, Admiralty) принимают решения на основе статических атрибутов: меток, ресурсной ёмкости. Они не учитывают динамические характеристики: текущий error rate кластера, p99 латентность, наличие активного инцидента. В результате нагрузка может направляться в деградирующий кластер, пока оператор вручную не вмешается.

4.3. Ручная и непроверяемая DR-процедура

Большинство практик DR в Kubernetes описаны в документации в виде инструкций (runbooks), но не автоматизированы в виде исполняемого кода. Velero обеспечивает backup/restore состояния, но не автоматизирует полный цикл failover: обнаружение деградации → переключение трафика → уведомление → восстановление исходного кластера → возврат трафика. Результат: время восстановления напрямую зависит от квалификации и доступности дежурного инженера.

4.4. Policy enforcement только на уровне кластера

Kyverno и OPA Gatekeeper работают как admission webhooks внутри отдельного кластера. Они не имеют информации о других кластерах федерации. Политика «этот workload должен размещаться только в кластерах с сертификацией PCI-DSS» не может быть выражена и исполнена в рамках существующих инструментов без написания custom-логики.

4.5. Отсутствие федерального каталога ресурсов

Для принятия решений о размещении meta-scheduler должен знать актуальный инвентарь кластеров: их метаданные, сертификации, доступные ресурсы, SLO-статус. Ни одно из рассмотренных решений не предоставляет структурированного, API-доступного каталога такого рода с проверками доверия.

5. Предлагаемые методы решения

5.1. Архитектура сквозного контура управления

Предлагаемое решение строится на принципе сквозного контура управления, в котором каждый компонент предоставляет данные следующему:

Наблюдаемость → SLO Engine → Каталог кластеров → Meta-Scheduler → DR Controller

Такой контур позволяет автоматически реагировать на изменение состояния инфраструктуры без ручного вмешательства. Ниже описаны ключевые компоненты.

5.2. Унифицированный слой наблюдаемости

Каждый кластер оснащается Prometheus + OpenTelemetry Collector. Для агрегации метрик на уровне федерации предлагается использовать Thanos Querier с sidecar-компонентами в каждом кластере. Это позволяет выполнять глобальные PromQL-запросы через единый endpoint и строить агрегированные SLO-метрики вида:

availability_global = sum(successful_requests) / sum(total_requests) по всем кластерам

Наряду с глобальными метриками, для каждого кластера вычисляются локальные SLI: error rate, p95/p99 latency, saturation (CPU/Memory utilization). Эти метрики являются входными данными для meta-scheduler'a.

5.3. Федеративный каталог (Registry Service)

Предлагается разработать REST API-сервис на FastAPI + PostgreSQL, выполняющий функцию реестра кластеров. Каждый кластер регистрируется в каталоге со следующими атрибутами: идентификатор, регион, провайдер, метки (labels), список поддерживаемых сертификаций (PCI-DSS, ISO 27001, GDPR-compliant), текущий SLO-статус (healthy / degraded / down), доступная ресурсная ёмкость.

Каталог периодически (каждые 30 секунд) обновляет статус кластеров, опрашивая Thanos Querier. Это обеспечивает актуальность данных для meta-scheduler'a.

5.4. Исполняемые политики размещения

Политики размещения формализуются в двух уровнях:

Уровень 1 — Hard constraints (обязательные ограничения):

- Географические ограничения (workload X может размещаться только в кластерах с region=eu-west)
- Ограничения по сертификациям (workload X требует кластер с certification=PCI-DSS)
- Ограничения по доступности ресурсов (минимум N CPU и M Gi памяти)

Уровень 2 — Soft constraints (предпочтения):

- Предпочтение кластера с наименьшим error rate
- Предпочтение кластера с наименьшей p95 latency
- Предпочтение кластера, не затронутого активным инцидентом

Hard constraints реализуются через Kyverno-политики на уровне каждого кластера (enforcement при попытке создания ресурса) и через фильтрацию в meta-scheduler'e на уровне федерации. Soft constraints реализуются через scoring-функцию в meta-scheduler'e.

5.5. Meta-Scheduler

Meta-scheduler — это контроллер (реализованный на Go с использованием controller-runtime), принимающий решение о том, в какой кластер федерации направить данный workload. Алгоритм работы:

6. Получить список workloads, ожидающих размещения (через Kubernetes watch API).
7. Для каждого workload извлечь из каталога список кластеров, прошедших hard constraint фильтрацию.
8. Для каждого допустимого кластера вычислить score = w1*(1 - error_rate) + w2*(1/latency_p95) + w3*(1 - saturation), где w1, w2, w3 — настраиваемые веса.
9. Выбрать кластер с максимальным score и создать в нём соответствующий ресурс (Deployment/Job).
10. Зафиксировать принятое решение в audit log каталога.

Для сравнительного анализа реализуется также «наивный» планировщик (round-robin по доступным кластерам). Оба варианта тестируются на одних и тех же сценариях отказов, метрики SLO сравниваются.

5.6. DR-контроллер

DR-контроллер — Kubernetes оператор, реализующий следующий цикл:

11. Мониторинг: периодически опрашивает Thanos/Prometheus для каждого кластера. Если error rate > порога или кластер недоступен более T секунд — объявляется инцидент.
12. Failover: уведомляет meta-scheduler о необходимости перенаправить новые workloads в здоровые кластеры. Обновляет DNS/Ingress (через ExternalDNS) для переключения трафика.
13. Recovery: после восстановления кластера выполняет smoke-тест (серия тестовых запросов), и при успехе постепенно возвращает трафик (traffic shifting через Istio или аннотации Ingress).
14. Документирование: записывает TTD (time to detect), RTO, RPO в формате, пригодном для анализа.

6. Детальный план на следующий семестр

6.1. Структура этапов

№	Задача	Срок	Ожидаемый результат
1	Развёртывание стенда: 3 кластера k8s (kind / k3d) + базовый тестовый сервис	Нед. 1–2	Работающий стенд, задокументированные конфигурации
2	Установка и настройка Prometheus + Thanos + Grafana в каждом кластере	Нед. 2–3	Глобальный дашборд метрик, агрегированный SLO
3	Интеграция OpenTelemetry для трассировки запросов между кластерами	Нед. 3–4	Распределённые трейсы, видимые в Jaeger/Grafana Tempo
4	Реализация федерального каталога (FastAPI + PostgreSQL)	Нед. 4–6	REST API с регистрацией кластеров, SLO-статусом, audit log
5	Разработка набора Kyverno-политик для enforcement hard constraints	Нед. 5–6	Политики размещения: регион, сертификация, ресурсы
6	Реализация прототипа meta-scheduler'a (Go, controller-runtime)	Нед. 6–9	Рабочий планировщик с scoring-функцией, наивная версия для сравнения
7	Реализация DR-контроллера: обнаружение → failover → recovery	Нед. 8–10	Автоматизированный DR-цикл с замером TTD/RTO/RPO
8	Серия экспериментов: 3 сценария отказов (chaos mesh)	Нед. 10–12	Таблицы метрик «до/после», графики деградации и восстановления
9	Сравнительный анализ: meta-scheduler vs наивный, с политиками vs без	Нед. 12–13	Количественные результаты, подтверждающие или опровергающие гипотезы

10	Написание финального отчёта и подготовка презентации	Нед. 13–15	Финальный отчёт НИР, слайды для защиты

6.2. Описание экспериментальных сценариев

Для объективной оценки предлагаемых механизмов планируется провести следующие контролируемые эксперименты:

Сценарий 1: Отказ одного кластера

С помощью Chaos Mesh принудительно выводится из строя один из кластеров (отключение API-сервера или всех подов). Измеряется: время обнаружения DR-контроллером (TTD), время переключения трафика (failover time), итоговый SLO за период инцидента. Сравниваются результаты: с DR-контроллером и без него.

Сценарий 2: Деградация сети (латентность + потери пакетов)

С помощью Chaos Mesh вносится сетевая задержка (500 мс) и потери пакетов (10%) для одного из кластеров. Измеряется реакция meta-scheduler'a: переключается ли он на здоровый кластер и насколько быстро? Сравниваются error rate и latency при умном vs наивном планировании.

Сценарий 3: Нарушение политики размещения

Предпринимается попытка разместить workload с метками, нарушающими активную Kyverno-политику (например, запуск на кластере без нужной сертификации). Измеряется: отклоняет ли система размещение, предлагает ли корректный кластер, фиксируется ли нарушение в audit log.

6.3. Технологический стек

Компонент	Технология	Обоснование выбора
Кластеры (стенд)	kind / k3d	Лёгкий запуск на одной машине, подходит для экспериментов
Метрики	Prometheus + Thanos	CNCF-стандарт, поддержка федерации через Thanos
Визуализация	Grafana	Де-факто стандарт, готовые дашборды для k8s

Трассировка	OpenTelemetry + Jaeger	Vendor-neutral, CNCF, поддержка Go и Python
Chaos Engineering	Chaos Mesh	CNCF-проект, поддержка k8s-native сценариев
Policy Enforcement	Kyverno	YAML-native, не требует Rego, проще для старта
Каталог	FastAPI + PostgreSQL	Лёгкий REST API, надёжная СУБД, простой деплой в k8s
Meta-Scheduler	Go + controller-runtime	Официальный SDK для k8s операторов
DR автоматизация	Go + controller-runtime	Единый язык для всех контроллеров
Traffic switching	Ingress-nginx + ExternalDNS	Доступен в kind/k3d, достаточен для стенда

7. Заключение

В рамках данного этапа НИР была сформулирована развёрнутая постановка задачи исследования, направленного на повышение надёжности федерации Kubernetes-кластеров. Проведён анализ существующих инструментов и подходов — KubeFed, Admiralty, OCM, Prometheus, Velero, Kyverno — и выявлены их ключевые ограничения:

- Разрозненность компонентов и отсутствие единого контура управления надёжностью;
- Отсутствие метрико-ориентированного выбора кластера в существующих federation schedulers;
- Неавтоматизированность DR-процедур на уровне полного цикла failover/recovery;
- Ограниченность политик размещения границами одного кластера;
- Отсутствие федерального каталога с актуальным SLO-статусом кластеров.

В ответ на выявленные ограничения предложена архитектура сквозного контура управления, включающего: унифицированный слой наблюдаемости на основе Thanos, федеративный каталог (FastAPI + PostgreSQL), систему исполняемых политик (Kyverno + meta-scheduler filtering), meta-scheduler с scoring-функцией и DR-контроллер с автоматизированным циклом failover.

Сформулирован детальный план экспериментальной реализации на 15 недель следующего семестра, включающий три контролируемых сценария отказов и количественное сравнение предлагаемых механизмов с базовыми подходами.

Ожидаемый научно-практический вклад работы: воспроизводимая методика измерения надёжности в мультикластерной среде, прототип meta-scheduler'a с верифицированными улучшениями SLO, набор исполняемых политик размещения и автоматизированный DR-план — артефакты, применимые в реальных production-инфраструктурах.

8. Список используемой литературы

1. Beyer B., Jones C., Petoff J., Murphy N.R. Site Reliability Engineering: How Google Runs Production Systems. — O'Reilly Media, 2016.
2. Beyer B., Murphy N.R., Rensin D.K., Kawahara K., Thorne S. The Site Reliability Workbook. — O'Reilly Media, 2018.
3. Burns B., Grant B., Oppenheimer D., Brewer E., Wilkes J. Borg, Omega, and Kubernetes. — ACM Queue, 2016. Vol. 14.
4. Kubernetes Documentation. Federation. — https://kubernetes.io/docs/concepts/cluster-administration/federation/ (дата обращения: май 2025).
5. CNCF. KubeFed v2 GitHub Repository. — https://github.com/kubernetes-sigs/kubefed
6. CNCF. Open Cluster Management Documentation. — https://open-cluster-management.io/
7. CNCF. Liqo Documentation. — https://docs.liqo.io/
8. Thanos Documentation. — https://thanos.io/tip/thanos/getting-started.md/
9. Prometheus Operator Documentation. — https://prometheus-operator.dev/
10. Kyverno Documentation. — https://kyverno.io/docs/
11. Chaos Mesh Documentation. — https://chaos-mesh.org/docs/
12. Velero Documentation. — https://velero.io/docs/
13. OpenTelemetry Documentation. — https://opentelemetry.io/docs/
14. Brendan Burns, Joe Beda, Kelsey Hightower. Kubernetes: Up and Running, 3rd Ed. — O'Reilly Media, 2022.
15. Google SRE. SLOs & Error Budgets — https://sre.google/workbook/implementing-slos/